

O Livro Lumen — v0.1 (esqueleto)

- > Da primeira linha ao `async`. Cada capítulo combina teoria curta,
- > código rodável em `examples/` e exercícios com gabarito.
- > Convenções de código: `docs/STYLE.md`. Referência: `docs/API.md`.

Como ler

- Capítulos 0–3: leia em ordem (~3h). É o "Lumen essencial".
- Capítulos 4–7: aprofunde por interesse; cada um é autocontido.
- Capítulos 8–12: avançado — compiler, backends e contribuição.
- Símbolo ■ rode: execute o arquivo indicado com `lumen run`.

Capítulo 0 — Boas-vindas e instalação

- O que é Lumen: linguagem com tipos verificados antes de rodar ("estática"), `match` que obriga a cobrir todos os casos ("exaustivo") e 3 formas de executar ("backends": `vm/c/wasm`).
- Instalação (script, Docker, fonte) + `lumen --version`.
- ■ rode `examples/001_hello.lum`. Anatomia do hello (linha por linha).
- Tour do repo: `spec/` (regras), `compiler/` (compilador), `stdlib/` (biblioteca pronta), `conformance/` (testes das regras).
- Exercícios: 3 (ver tutorial 01).

Capítulo 1 — Funções e tipos

- `fn`, parâmetros/retorno com tipo escrito ("anotados"), `return`, `let` (não muda) / `mut` (muda).
- Tipos: `i32` (inteiro), `f64` (decimal), `bool` (verdadeiro/falso), `char` (uma letra, `'x'`), `str/String` (texto), `()` ("nada").
- Operadores, ordem de cálculo ("precedência": `*` antes de `+`), `+` em `str` (junta), divisão `i32` (trunca) vs `f64` (com fração).
- ■ `002_fatorial.lum`, `003_fibonacci.lum`, `010_math.lum`.
- Exercícios: `div` com `Result`, conversões `str::to_int/from_int`.

Capítulo 2 — Controle de fluxo

- `if/else` (escolhe um de dois caminhos), `while` (repete enquanto verdade), `for in` (repete para cada item do intervalo).
- Intervalos ("ranges") `0..n` (exclui `n`) / `0..=n` (inclui `n`).
- Sem `loop/break/continue`: use `while` com condição.
- Erros como valor: `Result` (`Ok/Err`), `Option` (`Some/None`), `?` (repassa o erro), `panic` (só para bugs, encerra o programa).
- ■ estender `002` com `match` em `fat`.
- Exercícios: validador com `?` encadeado em 3 funções.

Capítulo 3 — Structs, enums e match

- `struct` (agrupa dados), `impl` (funções do tipo), `self` (o próprio valor); função da "fábrica" (`::`) vs. método do valor (`.`).
- Enums (um valor entre N opções) com/sem dados junto ("payload"); `Option/Result` como enums.
- `match`: valor fixo ("literal"), apelido ("binding"), `|` (ou), condição extra ("guard"), `_` (coringa), cobrir tudo ("exaustividade").
- ■ `004_match.lum`, `005_struct.lum`, `006_enum.lum`.
- Exercícios: Moeda, Triângulo, exaustividade quebrada de propósito.

Capítulo 4 — Coleções e strings

- `vec!` (cria lista), `map!` (cria dicionário), `vec::map/filter/len/push`, `map::get/has` (ler/testar chave).
- Quem é dono do valor ("ownership") em uma página: `str` (texto emprestado) vs `String` (texto próprio), `mut` (permite mudar), `+` junta.
- `std::str`: `split` (quebrar), `join` (juntar), `trim` (apurar), `upper` (maiúsculas), conversões; UTF-8 e caracteres.
- ■ `007_colecoes.lum`, `008_strings.lum`.
- Exercícios: contador de palavras com `Map`.

Capítulo 5 — IO, arquivos e CLI

- `io::{print, println, read_line, args}`, `fs::{read, write, existe}`.
- `env::var/cwd`, códigos de saída, `?` no `main`.
- ■ `009_io.lum`, `016_cli_json.lum`.
- Exercícios: cat mínimo; CLI com flag `--saida`.

Capítulo 6 — Testes e macros

- `#[test]`, `assert*`, `lumen test`, testes de `Result` com `?`.
- macro, higiene, `!` no uso; quando macro vs. `fn`.
- ■ `013_teste.lum`, `014_macro.lum`.
- Exercícios: macro `garante_algun!` para `Option`.

Capítulo 7 — HTTP, JSON, SQLite e async

- `http::get/post`, `json::parse/stringify`, `sqlite::{abrir, exec, query}`.
- `async fn`, `spawn`, `await`, `Handle`, executor determinístico.
- Projeto guiado completo (tutorial 05).
- ■ `011_http.lum`, `012_sqlite.lum`, `015_async.lum`.
- Exercícios: cache SQLite para o agregador.

Capítulo 8 — Tipos avançados e genéricos

- Genéricos (`Vec<T>` = lista de qualquer tipo `T`, `Map<K, V>`,
fns que servem para vários tipos), `trait` (só assinatura, sem corpo).
- `match` em tipos aninhados; erro como valor (`Result` encadeado).
- Erros comuns do checker e como ler cada mensagem.

Capítulo 9 — Bytecode e a VM

- Opcodes (`CONST/ADD/.../PRINT/HALT`), pool de constantes, `disasm`.
- `compile_expr + vm_run`: do AST ao resultado (ver `suite.py`, `BC-*`).
- Escrevendo seu primeiro opcode (receita passo a passo).

Capítulo 10 — Backends C e WASM

- `codegen_c` (`i32 ↔ int32_t`), `codegen_wasm` (`i32.*`).
- `lumen build --target`, FFI mínima, medindo binários.
- Casos `BE-*` da suíte como especificação executável.

Capítulo 11 — Stdlib por dentro

- Anatomia de um módulo `std::*`; documentando com `///`.
- Regenerando `API.md` com `docs/gen.py` (pipeline de docs).
- Receita: propor, implementar, testar e documentar função nova.

Capítulo 12 — Contribuindo e lançando

- Fluxo de PR, suíte 100% verde, `conformance_report.json`.
- Versionamento (`lumen.toml`), `infra/ci.yml`, `RELEASES.md`.
- Para onde levar o Lumen: roadmap e "boas primeiras issues".

Apêndices

- A — Gramática de bolso (tokens, expressões, precedência).
- B — Mensagens de erro mais comuns (tabela causa→correção).
- C — Diferenças Lumen × Rust × Python (mapa mental).
- D — Gabaritos dos exercícios.

Tutorial 01 — Instalação e Hello (~20 min)

> Nível 0. Ao final: toolchain instalada, primeiro programa rodando,
> lumen run/build/test/doc conhecidos.

1. Instalar

```
# via script (linux/macOS) ou Docker (ver infra/Dockerfile)
git clone <repo-lumen> && cd lumen
cargo build --release # ou: docker build -f infra/Dockerfile -t lumen .
lumen --version      # → lumen 0.1.0
```

2. Hello

Crie `ola.lum` (ou use `examples/001_hello.lum`):

```
/// Meu primeiro programa.
fn main() {
    println("Olá, Lumen!");
}

lumen run ola.lum
# → Olá, Lumen!
```

Três regras que já apareceram:

1. `fn main() { }` é a porta de entrada — todo programa tem uma; é por ela que a execução começa.
2. `println` imprime com quebra de linha no final (`\n` = "nova linha"; `print` não pula linha).
3. Toda instrução termina com `;` e o arquivo termina com uma quebra de linha (aperte Enter na última linha).

3. Compilar e testar

```
# "build" = gerar o programa final. Sem --target, roda na VM (máquina
# virtual do Lumen). Com --target c/wasm, gera código C ou WebAssembly.
lumen build ola.lum -o ola      # padrão: vm
lumen build --target c ola.lum  # backend C
lumen build --target wasm ola.lum # backend WASM (roda no navegador)
lumen test                     # roda as funções marcadas com #[test]
lumen fmt                      # formata o código (ver docs/STYLE.md)
lumen doc                      # gera docs via docs/gen.py
```

4. Exercícios

1. Imprima seu nome com `print` (sem `\n`) + `println`.
2. Quebre de propósito: tire o `;` e leia o erro do compilador.
3. Rode `python3 conformance/suite.py --filter runtime` e veja os

casos RT- * — são estes programas, como teste executável.

Próximo: 02_funcoes_tipos.md.

Tutorial 02 — Funções, tipos e controle (~45 min)

> Nível 1. Exemplos: 002_fatorial.lum, 003_fibonacci.lum, 010_math.lum.

1. Funções

```
/// Soma dois inteiros.  
fn soma(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

- Parâmetros e retorno com tipo escrito (a: i32). pub significa "pública" (pode ser usada de fora do arquivo).
- return explícito; só escrever o valor no final **não** retorna.
- Chamada: soma(2, 3) devolve 5.

2. Tipos básicos

Tipo	Exemplos
i32, i64	42, -7, 1_000
f64	3.14, 2.0 (/ em i32 trunca; em f64 não)
bool	true, false
char	'x' (aspas simples!)
str / String	"texto" (String coage para str)
()	() (unit, "nada")

```
fn main() {  
    // `let` cria uma variável imutável (não muda depois).  
    // O tipo `i32` (inteiro de 32 bits) foi descoberto sozinho.  
    let x = 5;  
    // `mut` libera a variável para mudar. Aqui o tipo foi escrito à mão.  
    let mut y: i32 = 0;  
    y = y + 1;  
    // Valor fixo que nunca muda: por convenção, nome em MAIÚSCULAS.  
    let max: i32 = 100;  
    println(str::from_int(x + y + max));  
}
```

3. Controle: if, for, while

```
fn mostra(n: i32) {  
    // `if/else` escolhe um dos dois caminhos.  
    if n < 0 {  
        println("negativo");  
    } else {  
        println("ok");  
    }  
}  
  
fn main() {
```

```

    mostra(-3);
    // `for i in 0..5` repete com i = 0, 1, 2, 3, 4 (o 5 fica de fora).
    for i in 0..5 {
        println(str::from_int(i));
    }
    let mut y: i32 = 0;
    // `while` repete enquanto a condição for verdadeira.
    while y < 10 {
        y = y + 1;
    }
    println(str::from_int(y));
}

```

Um "range" (intervalo) 0..5 exclui o 5; 0..=5 inclui o 5.

Ranges só funcionam com i32.

4. Erros com Result e ?

```

fn fat(n: i32) -> Result<i32, str> {
    if n < 0 {
        return Err("n negativo");
    }
    if n <= 1 {
        return Ok(1);
    }
    let sub = fat(n - 1)?; // Err aqui retorna da função
    return Ok(n * sub);
}

```

Regra prática: se o programa pode se recuperar do erro (ex.: número inválido), devolva Result e use ? para repassar. panic("msg") é só para bugs — ele encerra o programa.

5. Exercícios

1. Escreva fib_iter sozinho (gabarito em 003_fibonacci.lum).
 2. Faça div(a: i32, b: i32) -> Result<i32, str> que recusa b == 0.
 3. Rode python3 conformance/suite.py --filter tipos e leia os casos
- TIP-*: cada um é uma regra desta aula virando teste.

Próximo: 03_struct_enum_match.md.

Tutorial 03 — Struct, enum e match (~1h)

> Nível 2. Exemplos: 004_match.lum, 005_struct.lum, 006_enum.lum.

1. Struct + impl

```
/// Ponto no plano.
struct Ponto { x: f64, y: f64 }

impl Ponto {
    fn novo(x: f64, y: f64) -> Ponto {
        return Ponto { x: x, y: y };
    }
    fn norma(self) -> f64 {
        return math::sqrt(self.x * self.x + self.y * self.y);
    }
}

fn main() {
    let p = Ponto::novo(3.0, 4.0);
    println(str::from_float(p.norma())); // 5
}
```

- Construção: `Ponto { x: 1.0, y: 2.0 }` monta o valor; `p.x` lê o campo.

- `Ponto::novo(...)` é função da "fábrica" (chamada com `::`).

`p.norma()` é método do valor: recebe `self` (o próprio `p`) e usa ..

2. Enum com payload

```
enum Forma { Circulo(f64), Retangulo(f64, f64), Ponto }

fn area(f: Forma) -> f64 {
    return match f {
        Forma::Circulo(r) => 3.14159 * r * r,
        Forma::Retangulo(w, h) => w * h,
        Forma::Ponto => 0.0,
    };
}
```

Variantes com dados junto ("payload", ex.: `Circulo(1.0)`) e sem dados

(ex.: `Ponto`) convivem. `Option<T>` (`Some/None`, "tem valor / vazio")

e `Result<T, E>` (`Ok/Err`, "deu certo / deu erro") são enums prontos.

3. match a fundo

```
fn rotulo(n: i32) -> str {
    return match n {
        0 => "zero",
        // `|` significa "ou": vale para 1 ou 2.
        1 | 2 => "pequeno",
        // `v if ...` testa uma condição extra (chamada de "guard").
        v if v < 0 => "negativo",
        // `_` é o "coringa": pega todo o resto. Deixar o coringa no
        // final torna o `match` "exaustivo" (cobre todos os casos).
        _ => "médio",
    }
}
```

```
    };  
}
```

Ordem importa: o primeiro braço que combina vence. A condição extra (guard) não conta como cobertura — o `_final` continua obrigatório.

4. Exercícios

1. Adicione `Triangulo(b, h)` a `Forma` e cubra no match (o compilador acusa o braço faltante — teste!).
2. Crie `enum Moeda { Real(i32), Dolar(f64) } + em_reais(m: Moeda)`.
3. Rode `python3 conformance/suite.py --filter match e --filter struct_enum`: são os casos `MAT-*` e `ST-*`.

Próximo: `04_colecoes_strings_io.md`.

Tutorial 04 — Coleções, strings e IO (~1h)

> Nível 3. Exemplos: 007_colecoes.lum, 008_strings.lum,
> 009_io.lum, 010_math.lum.

1. Vec<T>

```
fn dobrar_um(x: i32) -> i32 {
    return x * 2;
}

fn eh_par(x: i32) -> bool {
    return x % 2 == 0;
}

fn main() {
    let nums = vec![1, 2, 3, 4, 5];
    // `vec![]` cria um vetor vazio; `vec::push` devolve o vetor com
    // o item novo (por isso guardamos de volta em `nomes`).
    let mut nomes = vec![];
    nomes = vec::push(nomes, "ana");
    // `vec::map` transforma cada item; `vec::filter` mantém só os que
    // passam no teste. Ambos devolvem um vetor novo e recebem o nome
    // de uma função pronta (sem parênteses).
    let dobrados = vec::map(nums, dobrar_um);
    let pares = vec::filter(nums, eh_par);
    println(str::from_int(vec::len(pares)));
    println(str::from_int(vec::len(dobrados)));
}
```

`vec::map/filter` recebem uma função e devolvem um vetor novo.

Prefira essa forma pronta ao `while` com `mut`; só troque pelo laço manual se medir e provar que ele é o gargalo (parte lenta).

2. Map<K, V>

```
fn main() {
    // `map!{...}` cria um dicionário chave → valor.
    let mut notas = map!{ "ana": 9, "bia": 7 };
    map::insert(notas, "cid", 10);
    // `get` devolve `Some(valor)` se a chave existe, ou `None`.
    match map::get(notas, "ana") {
        Some(v) => println(str::from_int(v)),
        None => println("sem nota"),
    }
}
```

`map::has(notas, chave)` testa se a chave existe; `map::remove(notas, chave)` tira a entrada e devolve o valor.

3. Strings UTF-8

```
fn main() {
    // `str::len` conta letras (caracteres), não bytes.
    println(str::from_int(str::len("olá"))); // 3
}
```

```
println(str::upper("lumen"));           // "LUMEN"
let partes = str::split("a,b", ",");    // vec!["a", "b"]
println(str::join(partes, " | "));      // junta com separador
println(str::trim(" x "));              // "x" (tira espaços)
println(str::from_int(42));             // "42" (número vira texto)
}
```

+ junta textos (str/String) em qualquer combinação.

4. IO e arquivos

```
use std::io;
use std::fs;

fn main() {
    // Cada operação pode falhar, então devolve `Result`.
    // `?` repassa o erro; o `match` no final trata tudo num lugar só.
    match io::read_line() {
        Ok(linha) => println("voce digitou: " + linha),
        Err(e) => println("erro no teclado: " + e),
    }
    match fs::read_arquivo("dados.txt") {
        Ok(texto) => println(texto),
        Err(e) => println("erro no arquivo: " + e),
    }
}
```

5. Exercícios

1. Leia dados.txt, conte linhas com `str::split(texto, "\n")`.
2. Monte um Map palavra→contagem (desafio: use match no get).
3. Rode a suíte `--filter stdlib` e confira STD-01/02/05/06.

Próximo: 05_projeto.md — o projeto guiado.

Tutorial 05 — Projeto guiado: agregador HTTP + SQLite (~2h30)

> Nível 4. Exemplos: 011_http.lum–016_cli_json.lum. Ao final: app

> concorrente com testes, macro e persistência — o "Lumen completo".

1. O que vamos montar

Um agregador que baixa itens de uma API (`http::get`), salva em

SQLite e responde consultas. Peças:

Peça	Arquivo-guia
HTTP + JSON	011_http.lum, 016_cli_json.lum
SQLite	012_sqlite.lum
Testes	013_teste.lum
Macro	014_macro.lum
Async	015_async.lum

2. Passo 1 — macro de erro com contexto

```
macro garante_ok(res, msg) {  
  match res {  
    Ok(v) => v,  
    Err(e) => panic(msg + ": " + e),  
  }  
}
```

Uso: `let db = garante_ok!(sqlite::abrir(":memory:"), "abrir");`

3. Passo 2 — sincronização async

```
async fn sincroniza(db: mut sqlite::Db, url: str) -> i32 {  
  let r = garante_ok!(http::get(url), "falha http");  
  return garante_ok!(sqlite::exec(db, r.corpo), "falha sql");  
}  
  
fn main() {  
  let mut db = garante_ok!(sqlite::abrir(":memory:"), "abrir");  
  garante_ok!(sqlite::exec(db, "CREATE TABLE item (id INT)", "ddl");  
  let h = spawn sincroniza(db, "https://api.exemplo.com/itens");  
  println("sincronizados = " + str::from_int(await h));  
}
```

`spawn` agenda a tarefa e devolve um "handle" (senha para buscar o resultado depois); `await` usa essa senha uma vez. As tarefas terminam na ordem em que foram criadas (executor de referência determinístico: sempre o mesmo resultado na mesma ordem).

4. Passo 3 — testes (inclusive de banco em RAM)

```
#[test]
fn testa_sincroniza() {
    let mut db = garante_ok!(sqlite::abrir(":memory:"), "abrir");
    garante_ok!(sqlite::exec(db, "CREATE TABLE item (id INT)", "ddl");
    assert_eq(sqlite::query(db, "SELECT * FROM item")?, vec![]);
}
```

`#[test]` pode usar `?`: se der `Err`, o teste falha. Banco `:memory:`

(na memória, sem arquivo) deixa o teste isolado — nada vaza de um teste para outro — e rápido.

5. Passo 4 — CLI que amarra tudo (ver `016_cli_json.lum`)

`payload(args)` é a função que monta o texto de saída a partir dos

argumentos (definida em `016_cli_json.lum` com `json::stringify`).

```
use std::io;
use std::env;
use std::fs;

fn payload(args: Vec<String>) -> String {
    return json::stringify(args);
}

fn main() {
    let args = io::args();
    match env::var("SAIDA") {
        Some(caminho) => {
            let r = fs::write_arquivo(caminho, payload(args))?;
            println("salvo");
        },
        None => println(payload(args)),
    }
}
```

6. Checklist final

- `[] lumen fmt limpo; ///` em todo item pub.
- `[] lumen test verde; python3 conformance/suite.py 50/50.`
- `[] docs/gen.py` regenerado se a API mudou.

Para onde ir: `examples/book/LIVRO.md` (capítulos 0→avançado) e `docs/API.md` (referência completa).

06 — Algoritmos básicos em Lumen

Ordenar é colocar em ordem; buscar é achar um item. Vamos usar o básico da linguagem: `vec` (lista), `while` (repete) e `match` (escolhe).

Bubble sort

A ideia: compare vizinhos e troque quem está fora de ordem. Repita até ordenar. Em Lumen não existe `v[i]`; lemos com `vec::get` (devolve `Some(valor)` ou `None`) e montamos o resultado com `vec::push`.

```
fn troca(v: Vec<i32>, i: i32, j: i32) -> Vec<i32> {
    let mut novo = vec![];
    let mut k: i32 = 0;
    let n = vec::len(v);
    while k < n {
        let atual = match vec::get(v, k) {
            Some(x) => x,
            None => 0,
        };
        if k == i {
            let outro = match vec::get(v, j) {
                Some(y) => y,
                None => 0,
            };
            novo = vec::push(novo, outro);
        } else {
            if k == j {
                let outro = match vec::get(v, i) {
                    Some(y) => y,
                    None => 0,
                };
                novo = vec::push(novo, outro);
            } else {
                novo = vec::push(novo, atual);
            }
        }
        k = k + 1;
    }
    return novo;
}

fn ordena(v: Vec<i32>) -> Vec<i32> {
    let mut res = v;
    let n = vec::len(res);
    let mut i: i32 = 0;
    while i < n {
        let mut j: i32 = i + 1;
        while j < n {
            let a = match vec::get(res, j - 1) {
                Some(x) => x,
                None => 0,
            };
            let b = match vec::get(res, j) {
                Some(x) => x,
                None => 0,
            };
            if a > b {
                res = troca(res, j - 1, j);
            }
        }
        i = i + 1;
    }
    return res;
}
```

```

        }
        j = j + 1;
    }
    i = i + 1;
}
return res;
}

```

Busca binária

Exige vetor ordenado. Compara o meio e descarta metade por iteração — $O(\log n)$. Veja `018_busca_binaria.lum`.

Exercícios

1. Transforme o bubble em `ordena_decrescente` (troque `a > b` por `a < b`).
2. Conte comparações com um contador mut (ex.: `let mut cont: i32 = 0;` some 1 a cada `if`) e imprima com `str::from_int`.
3. Implemente busca linear (olhe item por item) e compare o número de passos num vetor de 100 itens como `vec![1, 2, 3]`.

07 — Estruturas de dados

vec (lista), pilha, fila e mapa: as quatro peças básicas
(exemplos 019–021, 039).

Pilha e fila

Pilha é "último a entrar, primeiro a sair" (como pratos empilhados);

fila é "primeiro a entrar, primeiro a sair" (como fila de banco).

```
struct Pilha { itens: Vec }

impl Pilha {
    fn nova() -> Pilha {
        return Pilha { itens: vec![] };
    }
    fn push(self, v: i32) -> Pilha {
        return Pilha { itens: vec::push(self.itens, v) };
    }
    fn tamanho(self) -> i32 {
        return vec::len(self.itens);
    }
}

fn main() {
    let p = Pilha::nova();
    let p2 = p.push(10);
    println(str::from_int(p2.tamanho()));
}
```

Fila usa o mesmo vec com um número guardando a posição da "cabeça" em vez de remover do início (remover do início desloca todo o resto e custa proporcional ao tamanho — $O(n)$).

Mapa como dicionário

```
fn main() {
    // Dicionário nome → nota.
    let mut m = map!{ "ana": 9 };
    map::insert(m, "bia", 7);
    match map::get(m, "ana") {
        // `v` é número: converta para texto antes de imprimir.
        Some(v) => println(str::from_int(v)),
        None => println("ausente"),
    }
}
```

Exercícios

1. Implemente Fila com início: `i32` (índice, sem shift).
2. Faça `tabela_hash` com 8 baldes e colisões por lista.
3. Pagine um vetor de 10 em páginas de 3 (`039_paginacao.lum`).

08 — Recursão

Toda recursão precisa de: caso base + passo que encolhe a entrada.

```
fn fat(n: i32) -> Result<i32, str> {
    if n < 0 {
        return Err("n negativo");
    }
    if n <= 1 {
        return Ok(1);
    }
    match fat(n - 1) {
        Ok(v) => return Ok(n * v),
        Err(e) => return Err(e),
    }
}
```

Quando trocar por laço

Fibonacci chamando a si mesmo duas vezes explode (tempo "exponencial": dobra a cada nível); a versão com caderno de anotação ("cache", 033) ou com laço (003) cresce devagar ("linear"). Regra prática: recursão para formas que se repetem dentro de si (árvores, 023_arvore.lum); laço para contar.

Exercícios

1. soma_ate(n) nas duas versões (chamando a si mesma e com while); compare.
2. Busca em árvore binária (dica: cada filho pode existir ou não: use Option<Nodo> + match).
3. Busca por largura ("BFS": visitar por camadas) com fila (024_grafo_bfs.lum) — por que fila e não recursão? (Resposta curta: a ordem de visita é por camada, e a fila guarda "quem vem a seguir".)

09 — Complexidade sem mistério

Complexidade = "quantas operações o programa faz quando a entrada cresce".

n é o tamanho da entrada (ex.: quantos números há no vetor).

Padrão	Custo	Exemplo
um <code>while</code> até n	$O(n)$	soma em 034
dois <code>while</code> aninhados	$O(n^2)$	bubble sort
divide pela metade	$O(\log n)$	busca binária
tabela + reaproveita	$O(n)$	<code>fib_memo</code> em 033

Memoização

Trocar recomputação por memória: o vetor `memo` em `033_fib_memo.lum`

guarda -1 para "não calculado". De exponencial para linear com um vetor.

Exercícios

1. Meça (na mão) comparações do bubble vs. inserção em 10 elementos.
2. Mostre que `fib(30)` recursivo puro faria ~2M de chamadas.
3. Quando memoizar **não** ajuda? (dica: entradas que não se repetem)

10 — Strings na prática

str é UTF-8. Operações comuns (exemplos 008, 026–029):

```
fn main() {
    let s: str = " Olá, Lumen! ";
    // Sem espaços nas pontas e em maiúsculas.
    let limpa = str::upper(str::trim(s));
    println(limpa);
    let partes = str::split("a,b,c", ","); // vec!["a", "b", "c"]
    println(str::join(partes, "-"));
    // `str::to_int` pode falhar: devolve `Ok(número)` ou `Err(texto)`.
    match str::to_int("42") {
        Ok(n) => println(str::from_int(n)), // 42
        Err(e) => println("erro: " + e),
    }
    println(str::concat("a", "b")); // "ab"
}
```

Template simples

```
fn saudacao(nome: str) -> str {
    return str::concat("Olá, ", str::concat(nome, "!"));
}
```

Para múltiplas chaves, veja 028_template.lum (varre chaves/valores).

Exercícios

1. eh_palindromo (lê igual de trás para frente). Dica: compare a primeira letra com a última usando `str::len + while`, andando das pontas para o centro (não existe `s[i]`; use as funções `str::*`).
2. Contador de vogais: para cada letra `c`, use `match c { "a" | "e" | "i" | "o" | "u" => ... }`.
3. Leitor de chave=valor por linha (029_csv.lum como modelo: quebre por `"\n"`, depois cada linha por `"="`).

11 — Matemática e números

i32 (inteiro, ex.: 42), f64 (decimal, ex.: 3.14) e o módulo

math (exemplos 010, 031–034). Para usar, escreva `use std::math;`

no topo: isso libera os nomes `math::...`

```
use std::math;

fn main() {
    let raiz = math::sqrt(2.0);
    println(str::from_float(raiz));
    let pot = math::pow(2.0, 10.0);
    println(str::from_float(pot));
    // `math::fat` pode falhar (n negativo): devolve `Ok` ou `Err`.
    match math::fat(6) {
        Ok(v) => println(str::from_int(v)), // 720
        Err(e) => println(e),
    }
}
```

Inteiros vs. floats

/ de inteiros trunca (`7 / 2 == 3`); converta com `x as f64`

quando precisar de fração. Comparações encadeadas usam `match` com guards:

```
fn sinal(x: f64) -> str {
    return match x {
        v if v < 0.0 => "negativo",
        v if v > 0.0 => "positivo",
        _ => "zero",
    };
}
```

Exercícios

1. Juros compostos: `montante(c, i, n)` com `math::pow`.
2. `eh_primo` até 100 e a lista (`032_primalidade.lum`).
3. Média/desvio de um `vec` (`034_estatistica.lum`).

12 — Entrada e saída

O módulo `io` (exemplos 009, 029, 036):

```
use std::io;
use std::fs;

fn main() {
    // `println` escreve na tela. `io::read_line()` lê o teclado e
    // `fs::read_arquivo` lê um arquivo – ambos devolvem `Result`.
    println("lendo dados.txt...");
    match fs::read_arquivo("dados.txt") {
        Ok(txt) => println(txt),
        Err(e) => println(str::concat("erro: ", e)),
    }
    match io::read_line() {
        Ok(linha) => println("voce digitou: " + linha),
        Err(e) => println(str::concat("erro: ", e)),
    }
}
```

Arquivos e logs

- `fs::read_arquivo(caminho) / fs::write_arquivo(caminho, texto)` devolvem `Result` (use `match` ou `?`).
- Para logs que crescem (`036_log.lum`), abra uma vez e reuse a mesma referência ("handle": o valor que representa o arquivo aberto).
- CSV: leia tudo, `str::split` por `"\n"` (linhas) e por `" , "` (colunas) (`029_csv.lum`).

Exercícios

1. `copia(origem, destino)` com `?` para propagar erro.
2. Contador de linhas/palavras (`wc` simplificado).
3. Logger com níveis (`info`, `aviso`, `erro`) e timestamp.

13 — Programas de linha de comando

Todo CLI tem três partes: argumentos → configuração → execução

(016_cli_json.lum, 035_config.lum).

```
use std::io;

fn run(caminho: str) {
    println("abrindo: " + caminho);
}

fn main() {
    // `io::args()` devolve a lista de argumentos; a posição 0 é o nome
    // do programa. O `match` não aceita padrão de lista `[a, b]`, então
    // contamos e lemos por posição com `vec::len` + `vec::get`.
    let args = io::args();
    if vec::len(args) == 2 {
        match vec::get(args, 1) {
            Some(caminho) => run(caminho),
            None => println("uso: app <arquivo>"),
        }
    } else {
        println("uso: app <arquivo>");
    }
}
```

Config

```
struct Config { entrada: str, saida: str, verboso: bool }

fn padrao() -> Config {
    return Config { entrada: "-", saida: "-", verboso: false };
}
```

Exercícios

1. `--verboso` e `--saida <arq>`: percorra args com `while` e compare cada item com `==` (passo a passo, sem biblioteca de CLI).
2. Códigos de saída: 0 deu certo, outro número deu erro (o `main` dos exemplos imprime; o número de saída é definido pelo runner).
3. Leia o `lumen.toml` do projeto com `fs::read_arquivo` e imprima a linha do nome (seção `[package]`).

14 — Seu primeiro projeto

Juntando tudo: estrutura, módulos e build (05_projeto.md continua aqui).

```
meuapp/  
  lumen.toml  
  src/main.lum  
  src/calc.lum  
  testes/teste_calc.lum
```

lumen.toml

```
[package]  
nome = "meuapp"  
versao = "0.1.0"  
  
[tasks.bom]  
cmd = "python3 compiler/lumenc.py src/main.lum ir"
```

Build com o sistema da plataforma:

```
python3 tools/build/lumen_build.py
```

Módulos

Um módulo ("mod") é um arquivo .lum. use `std::io`; no topo libera os nomes daquele módulo. Convenções de nome: tipos com inicial maiúscula ("PascalCase", ex.: `Config`); funções com minúsculas e `_` ("snake_case", ex.: `padrao`). Função que pode falhar devolve `Result` — nunca sinalize erro com string vazia nem `-1` escondido ("mágico").

Exercícios

1. Separe `031_calculadora.lum` em `calc.lum` + `main.lum`.
2. Adicione task teste que roda a suíte de conformidade.
3. Publique no registry local (`tools/pkg/registry.py`).

15 — Concorrência com `async/await`

Concorrência = fazer dois downloads ao mesmo tempo em vez de um após o outro. `async fn` é uma função que pode pausar esperando a rede; `spawn` agenda a execução; `await` espera e pega o resultado (exemplos 015, 042).

```
async fn baixa(url: str) -> str {
  let r = http::get(url)?;
  return r.corpo;
}

fn main() {
  let a = spawn baixa("https://a.exemplo.com");
  let b = spawn baixa("https://b.exemplo.com");
  println(await a);
  println(await b);
}
```

Os dois downloads andam juntos; os `awaits` resolvem na ordem do `spawn`.

Regras

- `await` só funciona no valor que o `spawn` devolveu (a "senha"/handle). Cada handle é usado uma vez.
- `async` sem pausa dentro se comporta como função comum.
- `?` dentro de `async` transforma o erro em `Err`; trate com `match` após o `await` (ou repasse com `?` se a função atual devolve `Result`).

Exercícios

1. Baixe 3 URLs e imprima na ordem de chegada (dica: `await` alternado).
2. `com_retry` assíncrono (junte 038 + `spawn`).
3. Cache compartilhado entre tarefas (037_cache + `spawn`).

16 — Testes do seu código

Marque com `#[test]` e rode o runner (`013_teste.lum`):

```
fn soma(a: i32, b: i32) -> i32 {
    return a + b;
}

#[test]
fn testa_soma() {
    assert_eq(soma(2, 3), 5);
    assert_eq(soma(-1, 1), 0);
}

python3 tools/test/lumen_test.py .
```

Boa cobertura, barato

"Cobertura" = quanto do código os testes exercitam.

1. Um `#[test]` por função pública (a linha `#[test]` marca a função como teste; o runner a executa).
2. Casos de borda (extremos): lista vazia, zero, negativo, valor máximo.
3. Para `Result`: teste o caminho `Ok` e o caminho `Err`.

Exercícios

1. Testes para categoria (040) cobrindo as 5 faixas + negativo.
2. Teste de página (039) com página além do fim.
3. Teste que falha de propósito e leia o relatório JSON.

17 — Match avançado

Além do básico (`004_match.lum`): múltiplas opções num braço (`|`), condição extra (`if`, o "guard") e abrir o valor por dentro ("destruturar", ex.: `Forma::Circulo(r)` pega o `r` de dentro).

```
fn classifica(n: i32) -> str {
  return match n {
    0 => "zero",
    1 | 2 => "pequeno",
    v if v < 0 => "negativo",
    _ => "outro",
  };
}

fn area(f: Forma) -> f64 {
  return match f {
    Forma::Circulo(r) => 3.14159 * r * r,
    Forma::Retangulo(w, h) => w * h,
    Forma::Ponto => 0.0,
  };
}
```

Ordem importa: o primeiro braço que casa vence. `_` por último.

Guards (`if`) refinam sem aninhar `if/else`.

Exercícios

1. `http_status(code)`: 200–299 "ok", 400–499 "erro do cliente", 500–599 "erro do servidor" — use `v if ...` para faixas.
2. Abra um `Ok` com par dentro (padrão dentro de padrão).
3. Reescreva `040_validador.lum` trocando cada `if` por um braço de `match`.

18 — Enums modelam estados

Se o valor só pode ser uma de N coisas, é enum (006, 043, 047):

```
enum Estado { Novo, Pagamento, Enviado, Cancelado }
enum Acao { Pagar, Enviar, Cancelar }

fn transicao(e: Estado, a: Acao) -> Estado {
  return match e {
    Estado::Novo => match a {
      Acao::Pagar => Estado::Pagamento,
      Acao::Cancelar => Estado::Cancelado,
      _ => e,
    },
    _ => e,
  };
}
```

O compilador cobra exaustividade (cobrir todos os casos): esqueceu um braço, o match reclama. Assim, um estado inválido (ex.: "enviado sem pagar") nem tem como ser escrito — o match obriga a tratar cada caso.

Exercícios

1. Adicione `Estado::Reembolso` e veja onde o compilador aponta.
2. Semáforo (verde→amarelo→vermelho) como enum + `proximo()`.
3. `Papel { Admin, Editor, Leitor }` com permissões por variante (047).

19 — Erros como valores

Sem exceções: `Result<T, E> + ? (002, 009, 012, 038)`.

```
fn prepara(db: sqlite::Db) -> Result<i32, str> {
    // `?`: se der `Ok`, continua; se der `Err`, retorna o erro na hora.
    sqlite::exec(db, "CREATE TABLE u (nome TEXT)")?;
    sqlite::exec(db, "INSERT INTO u VALUES ('ana')")?;
    return Ok(1);
}

fn main() {
    match sqlite::abrir(":memory:") {
        Ok(db) => match prepara(db) {
            // `n` é número: converte antes de imprimir.
            Ok(n) => println(str::from_int(n)),
            Err(e) => println(str::concat("falhou: ", e)),
        },
        Err(e) => println(str::concat("não abriu o banco: ", e)),
    }
}
```

? abre o Ok ou devolve o Err na hora — evita `if` dentro de `if` ("pirâmide").

Regras

- Função que pode falhar retorna `Result`; nunca "código mágico".
- `main` decide: trata com `match` ou propaga com `?`.
- Teste os dois lados (tutorial 16).

Exercícios

1. `divide(a, b)` com `Err("divisão por zero") + retry (038)`.
2. Pipeline ler→parsear→salvar com `?` em cada etapa (048).
3. Converta um `panic!` hipotético em `Result` sem quebrar chamadores.

20 — Publicando seu pacote

Do `lumen.toml` ao registry local em 5 comandos:

```
# 1. inicia o projeto
python3 tools/pkg/lumen_pkg.py new meuapp
cd meuapp

# 2. declara e trava dependências
python3 ../tools/pkg/lumen_pkg.py add demo-pkg@'*'
python3 ../tools/pkg/lumen_pkg.py install

# 3. sobe o registry (outro terminal / outro Termux)
LUMEN_REGISTRY=http://127.0.0.1:8765 python3 ../tools/pkg/registry.py --port 8765

# 4. empacota e publica
python3 ../tools/pkg/lumen_pkg.py pack
python3 ../tools/pkg/lumen_pkg.py publish --registry http://127.0.0.1:8765

# 5. verifica a assinatura
python3 ../tools/pkg/lumen_pkg.py verify meuapp-0.1.0.lumepkg
```

O "registry" é o servidor onde o pacote fica guardado; "publicar" é enviar para lá. A assinatura prova quem criou o pacote: sem a variável `LUMEN_HMAC_KEY` (senha da assinatura) ela é de mentira ("demo", não use em produção — ver `LIMITACOES.md` item 4). Com a chave exportada (`export LUMEN_HMAC_KEY=...`), o `verify` confere de verdade. A lista de pacotes sai em `GET /index` (endereço `/index` no servidor).